CPU Scheduling Algorithms
Comparative Analysis Report

SRTF vs Round Robin — Four Scenario Study

Algorithms Compared:	SRTF (Shortest Remaining Time First) & Round Robin (RR)
Scenarios Analyzed:	A — Basic Mixed Workload | B — Quantum Sensitivity | C — Short Job Heavy | D — In
Metrics Evaluated:	Turnaround Time (TAT) | Waiting Time (WT) | Response Time (RT)

1. Introduction & Background

CPU scheduling is a fundamental component of operating system design. The choice of scheduling algorithm directly impacts system performance, process fairness, and user experience. This report compares two widely studied algorithms — **Shortest Remaining Time First (SRTF)** and **Round Robin (RR)** — across four distinct workload scenarios.

Algorithm	Type	Preemptive?	Core Principle
SRTF	Priority-based	Yes	Always run the process with the least remaining burst time
Round Robin	Time-sharing	Yes	Each process gets a fixed time quantum (q) in rotation

Metrics used: TAT (Turnaround Time) = Completion – Arrival | WT (Waiting Time) = TAT – Burst | RT (Response Time) = First CPU – Arrival

2. Scenario A — Basic Mixed Workload

A classic mixed workload where processes arrive at different times with varying burst lengths. RR quantum = 3. This scenario tests basic scheduling behaviour under non-uniform arrival and burst patterns.

2.1 Process Table

Process	Arrival Time	Burst Time
P1	0	5
P2	1	2
P3	2	7
P4	3	4

2.2 SRTF Gantt Chart

Timeline: 0 → 1 → 3 → 7 → 11 → 18

2.3 RR Gantt Chart (q = 3)

Timeline: 0 → 3 → 5 → 8 → 11 → 13 → 16 → 17 → 18

2.4 Performance Metrics

Metric	SRTF	RR (q=3)	Better
Avg TAT	8.25	11.75	★ SRTF
Avg WT	3.75	7.25	★ SRTF
Avg RT	3.25	2.50	★ RR

2.5 Analysis

SRTF delivers significantly lower Waiting Time (3.75 vs 7.25) and Turnaround Time (8.25 vs 11.75) by always choosing the process closest to completion. However, Round Robin offers a better Response Time (2.50 vs 3.25) because every process receives a CPU slice quickly, regardless of burst length. P2, the shortest job, completes earliest under SRTF — a clear demonstration of its short-job bias.

3. Scenario B — Quantum Sensitivity Case

Same process set as Scenario A but with a larger RR quantum (q = 5). This tests how increasing the time quantum affects RR performance and whether it converges toward FCFS behaviour.

3.1 Process Table

Process	Arrival Time	Burst Time
P1	0	5
P2	1	2
P3	2	7
P4	3	4

3.2 SRTF Gantt Chart

Timeline: 0 → 1 → 3 → 7 → 11 → 18

3.3 RR Gantt Chart (q = 5)

Timeline: 0 → 5 → 7 → 12 → 16 → 18

3.4 Performance Metrics

Metric	SRTF	RR (q=5)	Better
Avg TAT	8.25	10.00	★ SRTF
Avg WT	3.75	5.50	★ SRTF
Avg RT	3.25	4.50	★ SRTF

3.5 Analysis

With a larger quantum (q=5), RR loses its response-time advantage seen in Scenario A. SRTF now wins on all three metrics. A large quantum causes processes to run longer uninterrupted, essentially behaving like FCFS. This confirms that quantum size is a critical tuning parameter for RR: small quanta improve responsiveness but increase context-switch overhead; large quanta reduce overhead but degrade fairness and response time.

4. Scenario C — Short Job Heavy Case

All four processes have very short burst times (1–2 units) arriving close together. RR quantum = 3 (larger than most bursts). This is an ideal environment for SRTF and tests whether RR becomes equivalent when jobs are tiny.

4.1 Process Table

Process	Arrival Time	Burst Time
P1	0	2
P2	1	1
P3	2	1
P4	3	2

4.2 SRTF Gantt Chart

Timeline: 0 → 2 → 3 → 4 → 6

4.3 RR Gantt Chart (q = 3)

Timeline: 0 → 2 → 3 → 4 → 6

4.4 Performance Metrics

Metric	SRTF	RR (q=3)	Better
Avg TAT	2.25	2.25	★ Tie
Avg WT	0.75	0.75	★ Tie
Avg RT	0.75	0.75	★ Tie

4.5 Analysis

When all burst times fall below the RR quantum, both algorithms produce identical schedules and metrics. No process is ever preempted mid-execution, so SRTF's greedy selection produces the same execution order as RR's rotation. This scenario demonstrates that SRTF's advantage is greatest when there is high variance in burst lengths — when jobs are uniformly short, the distinction disappears.

5. Scenario D — Interactive Style Fairness Case

One long process (P1, burst=20) competes with three short processes (P2–P4, burst=3 each). RR quantum = 2. This simulates an interactive system where a background CPU-heavy task must coexist with quick foreground requests.

5.1 Process Table

Process	Arrival Time	Burst Time
P1	0	20
P2	0	3
P3	0	3
P4	0	3

5.2 SRTF Gantt Chart

Timeline: 0 → 3 → 6 → 9 → 29

5.3 RR Gantt Chart (q = 2)

Timeline: 0 → 2 → 4 → 6 → 8 → 10 → 11 → 12 → 13 → 29

5.4 Performance Metrics

Metric	SRTF	RR (q=2)	Better
Avg TAT	11.75	16.25	★ SRTF
Avg WT	4.50	9.00	★ SRTF
Avg RT	4.50	3.00	★ RR

5.5 Analysis

SRTF immediately recognises that P2, P3, and P4 are shorter than P1 and runs them to completion first. P1 then receives the CPU uninterrupted for 20 units. While this is highly efficient overall (WT=4.50), P1 suffers a very long wait. RR, by contrast, interleaves all four processes, giving P1 its first CPU slice immediately (RT=0) and ensuring short processes finish with low response time (RT=3.00). This makes RR far more suitable for interactive or real-time scenarios where responsiveness to all users matters more than raw throughput.

6. Cross-Scenario Comparative Summary

6.1 Metric Summary Across All Scenarios

Scenario	SRTF Avg WT	RR Avg WT	SRTF Avg RT	RR Avg RT	WT Winner	RT Winner
A – Mixed	3.75	7.25	3.25	2.50	SRTF	RR
B – Quantum	3.75	5.50	3.25	4.50	SRTF	SRTF
C – Short Jobs	0.75	0.75	0.75	0.75	Tie	Tie
D – Interactive	4.50	9.00	4.50	3.00	SRTF	RR

6.2 Analysis Questions

Q1: Which algorithm produced the lower average Waiting Time?

SRTF consistently produced lower average Waiting Time across all scenarios where the algorithms differed (Scenarios A, B, D). In Scenario C (all short jobs) they tied. SRTF's greedy approach of minimising remaining time ensures processes with little work left are completed quickly, reducing queue accumulation.

Q2: Which algorithm produced the lower average Response Time?

The answer is scenario-dependent. Round Robin achieved lower Response Time in Scenarios A and D (small quantum), where its time-sharing nature ensures every process receives a CPU slice rapidly. SRTF achieved lower (or equal) Response Time in Scenarios B and C. Overall, RR with a small quantum is superior for response time in mixed or interactive workloads.

Q3: Did SRTF favor short jobs more aggressively?

Yes — decisively. In Scenario D, SRTF ran P2, P3, and P4 (burst=3 each) entirely before touching P1 (burst=20), completing the three short jobs 20 time units before they would finish under a FCFS strategy. This is the defining characteristic of SRTF: it is essentially an online version of Shortest Job First (SJF), which is provably optimal for minimising average waiting time. The trade-off is potential starvation of long processes if short jobs keep arriving.

Q4: Which algorithm would you recommend for the tested workload?

For throughput-oriented or batch workloads (Scenarios A, B, C): SRTF is recommended. It minimises average waiting and turnaround time, completing more work per unit time. For interactive or time-sharing systems (Scenario D style): Round Robin with a well-chosen quantum is recommended. It ensures every user/process gets timely CPU attention and prevents any single long process from monopolising the CPU. If a single recommendation is required: SRTF, because it outperformed RR on Waiting Time in 3 of 4 scenarios and tied in 1, while RR only won on Response Time in 2 of 4 scenarios.

7. Conclusion

Algorithm that Performed Better Overall

SRTF performed better across the majority of scenarios and metrics. It achieved lower average Waiting Time and Turnaround Time in Scenarios A, B, and D. Scenario C resulted in a tie. Round Robin was superior only for Response Time in Scenarios A and D with a small quantum.

Metrics Better Under Each Algorithm

SRTF excelled at: Average Waiting Time (all non-tie scenarios), Average Turnaround Time (all non-tie scenarios), and Average Response Time when the quantum is large (Scenario B). RR excelled at: Average Response Time with small quanta (Scenarios A, D), reflecting its inherent fairness in time-sharing.

Main Trade-off Observed

The central trade-off is efficiency vs. fairness. SRTF maximises CPU utilisation and minimises average waiting time by aggressively prioritising short jobs — but at the cost of potentially starving long processes and offering poor response time to any process that is not the current shortest. Round Robin sacrifices overall throughput to guarantee that every process receives regular CPU attention, making it fairer but less efficient in batch contexts.

Which Algorithm Appeared Fairer in Practice

Round Robin is demonstrably fairer. By design, every process receives CPU time within at most (n-1)*q time units of arriving, regardless of burst length. In Scenario D, P1 (the long job) received its first CPU slice immediately at t=0 under RR, while SRTF kept P1 waiting until t=9. In real systems — especially those serving multiple users — this guarantee of bounded waiting is critical. SRTF's optimal average metrics are achieved at the expense of individual process fairness, which is an unacceptable trade-off in most interactive environments.

Final Recommendation: Use **SRTF** for batch/throughput workloads where minimising average waiting time is the priority. Use **Round Robin** (with a carefully tuned quantum) for interactive, real-time, or multi-user systems where response time and fairness are paramount.